

GPU acceleration of the myOCT tiled-scan reconstruction

Technical report — version `oct_recon_AF_v1`

Q3 OCT reconstruction project

dataset `10um_FOV_1` (Thorlabs GAN632, 864 tiles, 956 GB)

Generated September 26, 2026

Abstract

The legacy MATLAB reconstruction (`yOCTProcessTiledScan`, `myOCT`) of a $12 \times 9 \times 8$ -tile OCT volume costs 20.4 s per output plane, i.e. about 25.5 h serially for the 4500 planes of `10um_FOV_1`. The Python port `octrecon` (NumPy/Numba on CPU, CuPy/CUDA on GPU) re-implements the same mathematics with precomputed linear operators, a tile-major streaming I/O design and a fused CUDA pre-FFT kernel. It reaches 0.215 s/plane on an RTX 3080 Ti Laptop GPU (95× vs. legacy serial, 16.1 min extrapolated for the full volume) and 1.50 s/plane on the 8-core CPU (13.6×). The measured full-volume GPU reconstruction took 18.4 min end-to-end (83× faster than legacy serial). The GPU run is now limited by the USB SSD (≈ 0.9 GB/s): reading the raw data is the floor. Numerically the port matches the unmodified MATLAB code to 10^{-6} dB (float64) / 9.3×10^{-5} dB (float32), well below the 1.02×10^{-3} dB uint16 step, and reproduces the existing legacy TIFF bit-exactly (99.9–100 % identical pixels, max. 1 LSB) at dispersion $\beta = 8.949 \times 10^7$. Over the full volume (945,000,000 voxels) the production output is within 1.00 LSB of the legacy TIFF everywhere. Numbers marked [pending] are filled in automatically by `build_report.sh` once the corresponding result files exist.

Contents

1	Summary	3
2	System and data	4
3	How the raw data are stored	4
3.1	Unit of storage: one raw B-scan per file	4
3.2	Per tile: a standard Thorlabs OCITY container, unzipped	5
3.3	Thorlabs standard vs. myOCT convention	5
3.4	Tiles (“patches”) and their overlap	6
3.5	Consequences for reconstruction speed	6
4	The legacy pipeline	6
4.1	Call graph	6
4.2	Mathematics per raw B-scan	6
4.3	Where the legacy time goes	7
5	Acceleration analysis: what runs where, and why	7
5.1	Design principles	7
5.2	Raw I/O	8
5.3	Spectral k-linearisation (sinc5)	9
5.4	Window, dispersion and inverse FFT	9
5.5	Optical-path (field-curvature) correction	9
5.6	Stitching	10
5.7	Thread oversubscription on the CPU	10
5.8	Output	10
5.9	Why not everything on the GPU?	10

6	Benchmarks	10
6.1	Method	10
6.2	Throughput	11
6.3	Per-stage breakdown	12
6.4	Full-volume production run	12
6.5	Full-volume runs on the CPU and in legacy MATLAB	13
6.6	Resource usage during the full run	13
7	Validation and image quality	15
7.1	Stage-by-stage against the unmodified MATLAB code	15
7.2	Full plane (all 96 tiles, stitched)	15
7.3	Bit-exact reproduction of the legacy TIFF	15
7.4	Full-volume validation	15
8	Reconstruction parameters	17
8.1	What can be estimated automatically	17
8.2	z-scale caveat	17
9	Code layout, usage, limitations	17
9.1	Folder structure	17
9.2	How to run	18
9.3	Limitations	18
9.4	Next steps	18
A	Legacy per-script reference	18
B	Raw data format	19
B.1	Folder layout	19
B.2	Key <code>ScanInfo.json</code> fields	19
B.3	Binary files	20
B.4	Output format (legacy and port identical)	20

1 Summary

Headline numbers.

Implementation	s/plane	full volume	vs. legacy serial
Legacy MATLAB, serial (1 plane measured, extrapolated)	20.4	25.5 h	1×
Legacy MATLAB, full volume (default 8-worker parfor)	—	failed (OOM)	—
Python CPU, Numba 16 threads (extrapolated)	1.50	1.9 h	13.6×
Python CPU, full volume (measured, incl. TIFF)	1.29	1.6 h	15.8×
Python GPU, fused kernel (100-plane benchmark, extrapolated)	0.215	16.1 min	95×
Python GPU, full volume (measured, incl. TIFF)	0.245	18.4 min	83×

Measured full-volume runs. Python CPU: 1.6 h (GPU is 5.3× faster, outputs identical within 1 LSB). Original MATLAB: **killed by the out-of-memory killer after 3.9 min** with its default 8-worker pool on this 14 GB machine (§6.5); the legacy time is therefore the measured single-plane extrapolation.

The production GPU run reconstructed all 4500 planes in 18.2 min plus 15 s for the BigTIFF (18.4 min end-to-end), reading the 956 GB at an effective 877 MB/s.

What was accelerated and why it worked.

- **Spectral k-linearisation (sinc5)** — 77 % of the legacy runtime. The MATLAB code loops over the 2048 output samples of every B-scan and rebuilds the sinc filter each time. The filter depends only on the chirp, so it is precomputed once as a banded operator (≤ 14 taps/row) and applied by a fused kernel (Numba on CPU, CUDA `RawKernel` on GPU) that also performs the apodization subtraction, mean removal and the Hann×dispersion window. On the GPU fusion alone cut this stage from 33.76 s to 5.43 s per 9600 B-scans (6.2×).
- **FFT** — batched complex64 cuFFT over 25 000 A-lines per batch (GPU) or multithreaded pocketfft (CPU).
- **Optical-path correction and stitching** — per-B-scan `interp2` calls onto the full 35×6000 plane are replaced by a precomputed integer row-shift map and two small separable interpolation matrices applied as batched matrix products on the rows/columns that actually receive data; accumulators stay in GPU memory.
- **I/O and bookkeeping** — tile-major sequential reads into preallocated buffers with an 8-thread, 16-batch prefetch queue that overlaps disk reads with GPU compute; all geometry, headers and chirp are parsed once instead of once per plane.

After these changes the GPU spends most of its wall time waiting for the disk (12.78 s of 22.65 s in the fused benchmark, 56 %), the measured read rate is 911 MB/s ($\approx$ drive saturation), and the pipeline is *I/O bound*: the remaining lever is storage, not compute. The full run confirms this: the loop waited on the disk for 56 % of the reconstruction time and the GPU was busy ≈ 34 % on average.

Validation. Stage-by-stage agreement with the unmodified MATLAB code is at floating-point round-off; a full stitched plane agrees to 10^{-6} dB (float64) and 9.3×10^{-5} dB (float32); the existing legacy TIFF is reproduced bit-exactly at $\beta = 8.949 \times 10^7$ when the legacy double quantisation is enabled. **Full volume:** over all 945,000,000 voxels the production GPU output (default single quantisation) differs from the legacy TIFF by at most 1.00 LSB (0.00102 dB), mean 0.00019 dB; 100.0 % of voxels are within 1 LSB, 90.7 % have identical codes, and the NaN masks agree exactly (0 mismatches). The 1-LSB differences come from the legacy double quantisation, which the port omits by default (§7.4).

Recommended settings (configs/10um_FOV_1.yaml). `device: gpu, precision: float32, gpu_fused_kernel: true, batch_frames: 50, io_threads: 8, prefetch_batches: 16, dispersion_quadratic_term: 8.949e7, legacy_double_quantization: false`; on CPU-only machines set `OPENBLAS_NUM_THREADS=1` (§5.7).

2 System and data

Table 1: Hardware used for all measurements.

CPU	AMD Ryzen 9 6900HX, 8 cores / 16 threads
RAM	14 GB usable
GPU	NVIDIA RTX 3080 Ti Laptop, 16 GB GDDR6
Data disk	Crucial X10 8 TB USB SSD, exFAT; 911 MB/s to 961 MB/s measured under the threaded reader
Software	Python (conda env <code>oct_reconstruction</code>), NumPy/SciPy, Numba, CuPy (cuFFT, cuBLAS); legacy reference: MATLAB R2024a with the unmodified <code>Reconstruction_code_legacy/myOCT</code>

Table 2: Dataset `10um_FOV_1` (storage layout in §3, Appendix B).

Tiles	$12 (x) \times 9 (y) \times 8 (\text{depth}) = 864$ tiles of 1×1 mm, 500×500 A-lines
Raw B-scan	<code>Spectral{i}.data</code> : int16, 525 A-lines (25 apodization) $\times$ 2048 spectral px = 2.15 MB
Raw volume	$864 \times 500 = 432,000$ B-scan files, 929 GB of spectra, 956 GB on disk
Output	$4500 (y) \times 35 (z) \times 6000 (x)$, 2 μm isotropic, uint16 BigTIFF (+ float32 dB <code>.npy</code>)
Work per plane	96 raw B-scans ($12 x\text{-tiles} \times 8 \text{ depths}$) $\rightarrow$ one 35×6000 plane

An important consequence of the geometry: each output y -plane needs one B-scan from each of 96 tile folders, and every raw byte is used exactly once. The minimum work is therefore “read 956 GB once”, which at 0.95 GB/s is about 16–17 minutes. This I/O floor is drawn in Fig. 3.

3 How the raw data are stored

The storage layout determines the I/O pattern and therefore the achievable speed, so it is described here in full (source: docs/02 §10; acquisition code `Scanningcode/.../ThorlabsImagerPython/thorlabs_imager_oct.py`, `yOCTScan3DVolume`, l. 167–312, and `ThorlabsImager/yOCTScanTile.m`).

3.1 Unit of storage: one raw B-scan per file

The unit of storage is **one raw B-scan per file** — not one A-scan per file and not one tile per file. Each `Spectral{i}.data` is exactly one frame as delivered by the Thorlabs SpectralRadar SDK:

- 525 consecutive camera lines $\times$ 2048 spectrometer pixels, int16 little-endian, A-line major (`raw[525][2048]`, 2,150,400 bytes);
- lines 0–24 are the 25 **apodization** lines: the galvo is parked at `ApoVoltage` so the beam sees no sample, giving a reference spectrum recorded before *each* B-scan (`ApodizationType=EachBScan`);
- lines 25–524 are the 500 **scan A-lines** $x = 0 \dots 499$ of that B-scan, each a contiguous 2048-sample spectrum;
- values are raw 12-bit camera counts in 16-bit words: no dechirp, FFT or background subtraction has been applied;
- i is the tile-local y frame (0-based), $i = (y - 1) \cdot \text{nBScanAvg} + (\text{avg} - 1)$, so B-scan repeats would be consecutive (here `nBScanAvg` = 1).

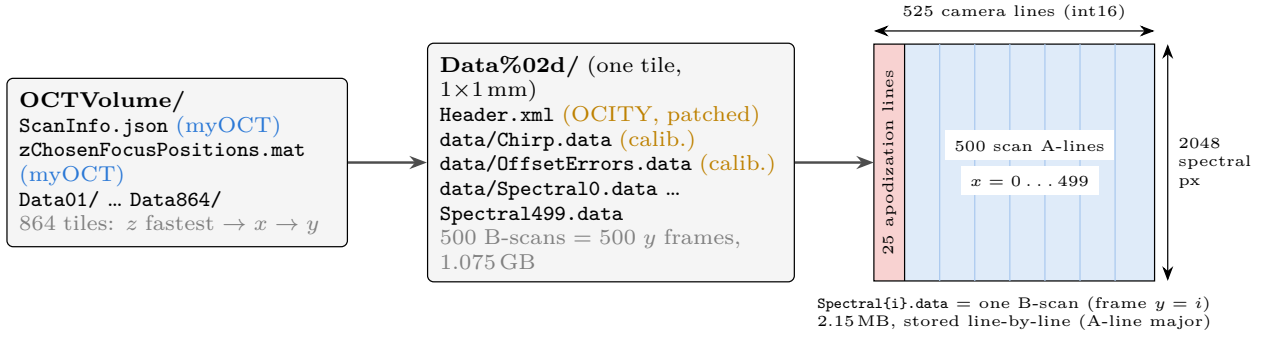


Figure 1: Storage hierarchy: volume folder → tile folder (one Thorlabs OCITY container, unzipped) → one file per raw B-scan → [25 apodization lines | 500 scan A-lines] × 2048 spectral pixels. Blue labels: myOCT convention; orange: Thorlabs OCITY content.

3.2 Per tile: a standard Thorlabs OCITY container, unzipped

How a tile folder is produced during acquisition (yOCTScan3DVolume):

1. Scan pattern `create_volume_pattern(rangeX, 500, rangeY, 500, EACH_BSCAN apodization, FRAME_BY_FRAME)`, shifted to the tile centre (probe `DynamicOffsetX` + galvo-delay correction ⇒ `CenterX=0.01735`).
2. Acquisition is `ASYNCFINITE`: the SDK delivers B-scans one at a time; each is added to an `OCTFile(OCITY)` as `data\Spectral{bscan_idx}.data` in arrival order. A lost frame aborts the tile.
3. `save_calibration` adds `data/Chirp.data` (float32[2048], pixel → k mapping) and `data/OffsetErrors.data` (float32[2048], unused by reconstruction); `set_metadata` fills `Header.xml` (device, probe, pattern, processing defaults, scan time).
4. The container is written as `scan.oct` — a **ZIP archive**, the standard ThorImage OCT file — then **extracted in place and deleted**, so MATLAB can read the parts directly.
5. `_fix_header_xml_for_matlab` patches `Header.xml` (Raw DataFile SizeZ/SizeX/SizeY, ApoRegion/ScanRegion, Image/SizePixel/SizeY, SpeckleAveraging/SlowAxis) because the SDK header does not match what `yOCTLoadInterfFromFile` expects.

Raw data are uncompressed (`CompressionLevel 0`; `unzipOCTFile=false` for Gan632), so the 956 GB on disk is essentially the raw camera stream ($100 \text{ kHz} \times 2048 \text{ px} \times 2 \text{ B} \approx 410 \text{ MB/s}$ during acquisition).

3.3 Thorlabs standard vs. myOCT convention

Table 3: Which parts of the layout come from where.

Item	Convention	Notes
Header.xml + data/*.data inside a tile	Thorlabs OCITY (.oct = ZIP)	readable by ThorImage OCT if re-zipped; header patched by <code>_fix_header_xml_for_matlab</code>
Spectral{i}.data per-frame naming	Thorlabs SpectralRadar SDK	raw-frame naming inside OCITY files
Chirp.data, OffsetErrors.data	Thorlabs calibration	only the chirp is used
Data%02d tile folders (Data01...Data864)	myOCT (yOCTScanTile.m)	%02d ⇒ 3 digits beyond 99; one OCITY folder per tile
Tile order $z \text{ fastest} \rightarrow x \rightarrow y$	myOCT	<code>meshgrid(xCenters, zDepths, yCenters)(:);</code> folder = $z_i + 8x_i + 96y_i + 1$
ScanInfo.json	myOCT	grid, per-folder tile centres <code>gridXcc/Ycc/Zcc</code> , full probe .ini, pixel size, RI; written after all tiles
zChosenFocusPositions.mat, .png	myOCT post-processing	from <code>yOCTMeasureFocusDrift</code> , not acquisition

3.4 Tiles (“patches”) and their overlap

The stage moves between tiles (rotated by `Oct2StageXYAngleDeg = -0.77°`); the galvos only scan within a 1×1 mm tile. Tiles do **not** overlap in x/y (tile step = tile size = 1 mm), so every output column/row comes from exactly one (x, y) tile. In z , the 8 depths ($-30 \dots +40 \mu\text{m}$, $10 \mu\text{m}$ steps) overlap heavily within the ≈ 1.47 mm A-scan and are **blended** by the focus-weighted stitching (§4, step 7–9).

3.5 Consequences for reconstruction speed

There are 432,000 B-scan files of 2.15 MB. The natural read unit is a tile (500 sequential files, 1.07 GB). One output y -plane needs frame i from each of the 96 tiles ($12x \times 8z$) of its tile row. The legacy stitcher reads per *output plane* (one file from each of 96 folders per plane); the port reads *tile-major* and streams at the drive’s limit (§5.2).

4 The legacy pipeline

The legacy entry point is `Demo_ScanAndProcess3D.m`, which calls `yOCTProcessTiledScan.m` with `interpMethod='sinc5'`, `focusSigma=10`, `cropZRange_mm=[-0.03 0.04]`, `outputFilePixelSize_um=2` and the focus file `zChosenFocusPositions.mat`. The analysis below was verified by running the unmodified legacy code (`validation/matlab/make_reference_plane.m`) and reproducing every intermediate in Python.

4.1 Call graph

```
yOCTProcessTiledScan
|-- awsReadJSON(ScanInfo.json), yOCTProcessTiledScan_createDimStructure (header, chirp, z axis)
|-- parfor yI = 1:4500 (each OUTPUT y plane)
|   |-- yOCTProcessTiledScan_getScansFromYFrame (re-runs createDimStructure every plane!)
|   |-- for xxI = 1:12, for zzI = 1:8 (96 tiles)
|       |-- yOCTLoadInterfFromFile(one frame) -> imageDatastore per file, apod. subtraction
|       |-- yOCTInterfToScanCpx -> yOCTEquispaceInterf('sinc5') (MATLAB loop over 2048 samples)
|       |   |-- Hann/rms * exp(-i*beta*(k-k0)^2), ifft, keep 1024 bins
|       |   |-- abs, yOCTOpticalPathCorrection (meshgrid + interp2 'nearest')
|       |   |-- yOCTProcessTiledScan_factorZ (Gaussian focus weight)
|       |   |-- interp2 x2 (data*weight, weight) onto the full 35x6000 output plane
|       |-- weighted mean -> mag2db -> yOCT2Tif(partialFileMode=2) (one tif+json per plane)
|-- yOCT2Tif(partialFileMode=3): global clim, re-quantise every plane, write BigTIFF
```

4.2 Mathematics per raw B-scan

With $N = 2048$ spectral pixels, $n_X = 500$ A-lines, 25 apodization lines and $n_Z = 1024$ depth pixels:

1. **Read** $\text{raw} \in \mathbb{Z}^{525 \times 2048}$ (int16, A-line major).
2. **Apodization subtraction** $I = \text{raw}[25:] - \overline{\text{raw}[25]}$ (mean over the 25 apodization A-lines).
3. **k-linearisation (sinc5)**: $k_j = 2\pi/\lambda_j$, normalised sample positions $n_j = (k_j - k_{\min})/(k_{\max} - k_{\min})(N - 1)$, targets $n_{q_i} = N - 1 - i$,

$$I_i^{\text{eq}} = \sum_{|n_{q_i} - n_j| < 5} \text{sinc}(n_{q_i} - n_j) (I_j - \bar{I}) + \bar{I},$$

with $\bar{I}$ the per-A-line spectral mean.

4. **Window + dispersion**: $W(k) = \frac{\text{hann}(N)}{\text{rms}(\text{hann})} e^{-i\beta(k^{\text{eq}} - \overline{k^{\text{eq}}})^2}$.
5. **FFT**: $S = \text{ifft}(I^{\text{eq}}W)[1:1024]$, $A = |S|$.
6. **Optical-path correction**: $A_c[z, x] = A[\text{nearest}(z + P(x, y)), x]$ with the field-curvature polynomial $P = p_1x + p_2y + p_3x^2 + p_4y^2 + p_5xy$; samples outside the depth range are invalid.
7. **Focus weight**: $F[z, x] = (e^{-(z-f)^2/(2\sigma)^2} + e^{-4.5}) \text{valid}[z, x]$, $f = 433$, $\sigma = 10$.
8. **Placement + stitching**: $\text{num} += \text{interp2}(A_cF)$, $\text{den} += \text{interp2}(F)$ (linear, 0 outside) for all 96 tiles of the plane.
9. **Normalise**: $\text{den} < e^{-4.5} \Rightarrow \text{NaN}$; $\text{dB} = 20 \log_{10}(\text{num}/\text{den})$.
10. **Write**: uint16 with the global clim.

4.3 Where the legacy time goes

Table 4: Legacy MATLAB timing of one output plane (96 B-scans), R2024a, 8 threads (docs/01 §7).

Stage	Time [s]	Share
yOCTInterfToScanCpx (sinc5 loop + FFT)	15.74	77 %
yOCTLoadInterfFromFile (96 files, one datastore per file)	1.56	8 %
createDimStructure + getScansFromYFrame (repeated per plane)	2.43*	–
yOCTOpticalPathCorrection	1.04	5 %
2×interp2 onto the full 35×6000 plane, ×96	0.71	3.5 %
Plane total	20.4	

* `getScansFromYFrame` (1.0 s) runs every plane and calls `createDimStructure` again inside.

$4500 \times 20.4 \text{ s} \approx 25.5 \text{ h}$ serial. The `parfor` over output planes runs 8 MATLAB workers, but they share the same 8 physical cores that the serial run already uses through multithreaded BLAS/FFT, and each worker re-reads headers per plane, so it cannot scale close to $8\times$. A full-volume run of the original MATLAB reconstruction on the default 8-worker pool was attempted: **failed (OOM)** (killed by the kernel OOM killer after 3.9 min, §6.5).

5 Acceleration analysis: what runs where, and why

This section is the core of the report. For each stage it gives the legacy implementation, the reason it is slow, the CPU optimisation, the GPU implementation and the measured effect. Table 5 is the one-page overview; Fig. 2 shows how the stages are chained in the port.

5.1 Design principles

1. **Precompute everything that does not depend on the data.** The sinc weights, the complex window, the optical-path shift map and the stitching interpolation matrices depend only on the chirp, `ScanInfo.json` and the parameters. The legacy code recomputes them for each of the 432,000 B-scans (and parses headers/JSON/chirp for every plane); the port builds them once in `geometry.py/stitching.py` (host, float64) and uploads them.
2. **Turn loops and `interp*` calls into linear operators.** k-linearisation becomes a banded sparse matrix, stitching becomes two small dense matrices; both map onto gather/FMA kernels and GEMM.
3. **Process tile-major, not plane-major.** Output rows of 500 planes are accumulated in memory so each tile’s 500 files are read sequentially once.
4. **Batch:** 50 B-scans = 25,000 A-lines per GPU launch keeps kernels and cuFFT large enough to saturate the GPU.
5. **Same code on both devices.** Every routine takes an array module `xp` (NumPy or CuPy, `backend.py`); only the pre-FFT kernel has device-specific versions (`cpu_kernels.py`, `gpu_kernels.py`).

Table 5: Stage-by-stage overview. Colour of the “runs on” column: **CPU**, **GPU**.

Stage	Legacy MATLAB	CPU port	GPU port	Runs on (GPU mode)
Raw I/O	<code>imageDatastore</code> per 2 MB file; plane-major access to 96 folders; header/JSON/chirp re-parsed each plane	tile-major <code>readinto</code> into preallocated int16 buffers; 8 threads, 16-batch prefetch	same host reader; batch copied to device (<code>cp.asarray</code>)	CPU threads (overlapped)
Apodization subtraction	per file in loader	fused into Numba kernel	fused into CUDA kernel (apod mean via CuPy reduction)	GPU
sinc5 k-linearisation	MATLAB loop over 2048 output samples, filter rebuilt each time (77% of runtime)	banded operator (≤ 14 taps), Numba <code>prange</code> fused kernel	CUDA <code>RawKernel</code> , 1 thread per output sample, fused	GPU
Window + dispersion	elementwise per B-scan	fused into kernel (precomputed complex window)	fused into kernel	GPU
IFFT (2048, keep 1024)	<code>ifft</code> per B-scan	<code>scipy.fft</code> (pocketfft), all cores	batched cuFFT, complex64, 25,000 lines	GPU
Magnitude	<code>abs</code>	NumPy	CuPy elementwise	GPU
Optical-path corr.	<code>meshgrid</code> + <code>interp2('nearest')</code> per B-scan	precomputed integer shift map + validity bounds; gather on needed rows only	same, <code>take_along_axis</code> on device	GPU
Focus weight	<code>factorZ</code> per B-scan	precomputed vector	precomputed vector	GPU
Stitching	$2 \times \text{interp2}$ of every B-scan onto full 35×6000 plane	separable W_z, W_x restricted to hit rows/cols; batched matmul (OpenBLAS, 1 thread)	batched matmul (cuBLAS); accumulators resident in VRAM	GPU
Finalise	weighted mean, <code>mag2db</code> per plane	NumPy	CuPy, then one D2H copy per row	GPU
Output	two quantisations; 100 KB JSON on all 4500 pages; one tif per plane then merge	float32 memmap, global clim, single quantisation, JSON on page 0	same	CPU

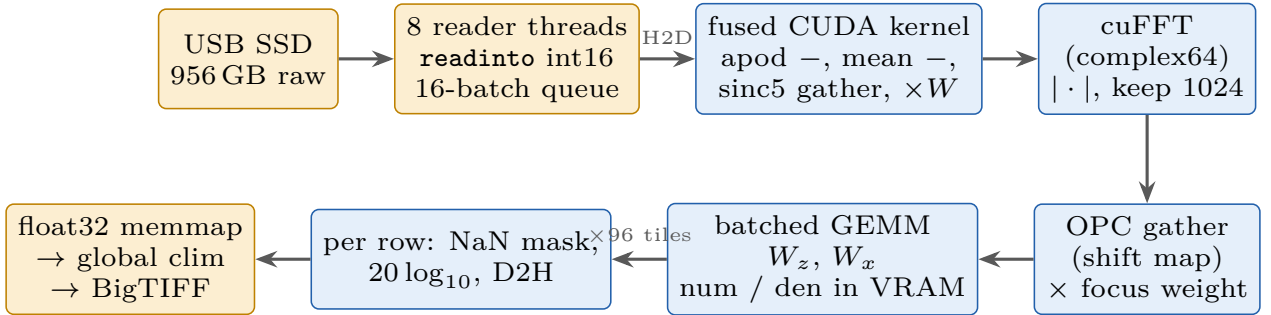


Figure 2: Data flow of the GPU port for one y -tile row (500 output planes, 96 tiles $\times$ 500 B-scans, processed in batches of 50). Orange = CPU, blue = GPU. Reading is overlapped with GPU compute by the prefetch queue.

5.2 Raw I/O

Legacy. `yOCTLoadInterfFromFile_ThorlabsData.m` creates a new MATLAB `imageDatastore` for every 2 MB `Spectral*.data` file. The `parfor` runs over *output planes*, so each plane opens one file in each of 96 tile folders; consecutive reads of the same folder are 4500 plane-iterations apart. In addition `getScansFromYFrame` re-parses `ScanInfo.json`, `Header.xml` and the chirp and rebuilds the dimension structure for every plane (≈ 2.4 s/plane together with `createDimStructure`).

Port (CPU and GPU identical). `io/thorlabs.py:read_bscans_into` opens each file unbuffered and `readintos` it directly into a preallocated int16 array (no intermediate copies, no decode). `pipeline.py:process_row` iterates *tile-major* over the 96 tiles of a y -row and reads each tile’s frames in order, submitting batches to a `ThreadPoolExecutor` (8 threads) with a queue of 16 outstanding batches ($16 \times 50 \times 2.15$ MB $\approx$

1.7 GB in flight). Header, chirp and geometry are read once in `Reconstructor.__init__`. Python threads work here because file reads release the GIL.

Effect. Loading cost per plane drops from ≈ 1.56 s (+2.4 s bookkeeping) to effectively zero *compute* time: reading is hidden behind GPU work until the disk saturates. The fused GPU benchmark reads at 911 MB/s (911 MB/s to 961 MB/s across runs, Crucial X10 saturation), which puts the floor for 956 GB at ≈ 16 –17 min.

5.3 Spectral k-linearisation (sinc5)

Legacy. `y0CTEquispaceInterf.m` (myInterp, sinc branch):

```
for i = 1:length(xq)           % 2048 output samples
    filt = sinc(nq(i)-n);      % 2048 sinc evaluations
    ii = abs(nq(i)-n) < nSamples; % ~10 samples used
    out(i,:) = sum((v(ii,:) - mv).*repmat(filt(ii),[1 size(v,2)]),1);
end
```

Per B-scan this is 2048 interpreted iterations, each evaluating 2048 sines and a mask, allocating temporaries and `repmat`-ing the filter across 500 A-lines — although the filter depends only on the chirp. It accounts for 15.7 of 20.4 s per plane (77 %).

Precomputation. `geometry.py:build_spectral_geometry` evaluates the same sinc weights once and stores them as a banded operator: index and weight arrays of shape (N, T) with $T \leq 14$ taps per output sample (padding taps have weight 0). The Hann window, its RMS normalisation and the dispersion phase are folded into one complex vector $W \in \mathbb{C}^{2048}$.

CPU. `cpu_kernels.py:fused_pre_fft` (Numba, `parallel=True`, `fastmath=False`) parallelises over A-lines with `prange`. For each A-line it subtracts the apodization mean, computes and removes the spectral mean, gathers the ≤ 14 taps per output sample, re-adds the mean and multiplies by W — all on a 2048-sample buffer that stays in L1/L2 cache, and in the same operation order as the reference, so results are numerically equivalent. The pure-NumPy fallback (`SpectralProcessor.linearise`) does the same with 14 vectorised `take`+FMA passes over the full batch, which creates many full-size temporaries; end-to-end it is $2.7\times$ slower than the final Numba configuration (4.14 vs. 1.50 s/plane).

GPU. `gpu_kernels.py` is a CuPy `RawKernel` with *one thread per output spectral sample* (grid-stride loop over $B \times 500 \times 2048$ samples). Two cheap CuPy reductions first produce the apodization mean per B-scan and the per-A-line mean ($\overline{\text{raw}} - \text{apod}$); the kernel then reads the int16 raw samples directly, applies the subtraction, the T -tap sinc gather and the complex window, and writes the complex64 spectrum ready for cuFFT. Neighbouring threads touch neighbouring raw samples, so the gathers are coalesced and served from L1/L2.

Why fusion matters on the GPU. The unfused CuPy path is numerically fine but memory-bound: `astype`, two means, 14 gather passes and 14 multiply-adds each stream a $50 \times 500 \times 2048$ float32 array (205 MB) through global memory, plus a complex multiply. The fused kernel reads the 107 MB int16 input once and writes the 410 MB complex64 output once. Measured over 100 planes (9600 B-scans): spectral stage $33.76 \text{ s} \rightarrow 5.43 \text{ s}$ ($6.2\times$), wall time $39.33 \text{ s} \rightarrow 22.65 \text{ s}$ ($1.74\times$), after which the disk becomes the bottleneck.

5.4 Window, dispersion and inverse FFT

Legacy. `y0CTInterfToScanCpx.m` builds the Hann window and dispersion phase per call (options are even parsed with `eval`), multiplies and calls `ifft` on one B-scan at a time.

CPU. Window is fused into the Numba kernel; the IFFT uses `scipy.fft.ifft(..., workers=os.cpu_count(), overwrite_x=True)` (pocketfft, multithreaded, same $1/N$ normalisation as MATLAB).

GPU. Window is fused into the CUDA kernel; the IFFT is a single batched cuFFT call on a complex64 array of $25,000 \text{ lines} \times 2048$ (50 B-scans), followed by slicing the first 1024 bins and `abs`. cuFFT plans are cached after the warm-up batch. complex64/float32 is the production precision because its error ($\approx 10^{-4}$ dB) is $10\times$ below the output quantisation step (§7); the RTX 3080 Ti Laptop has far lower float64 throughput, and float64 would double memory traffic.

5.5 Optical-path (field-curvature) correction

Legacy. `y0CTOpticalPathCorrection.m` builds a `meshgrid` and calls `interp2(..., 'nearest')` for each B-scan at $z + P(x, y)$, marking out-of-range samples invalid.

Port. Because the tile z grid is uniform, nearest-neighbour resampling at $z_r + P$ is exactly a row shift $r + \text{round}(P/\Delta z)$, and validity is $0 \leq r + P/\Delta z \leq n_z - 1$. `OpticalPathCorrection.build` computes the integer shift map and $P/\Delta z$ once for all 500×500 (y, x) positions — the map is identical for every tile. At run time `TileStitcher.contribute` applies it as a single `take_along_axis` gather over only the rows $[r_0, r_1)$

that can reach the cropped output z range (a few tens of the 1024 rows). Tie-breaking ($\text{floor}(x + 0.5)$) matches MATLAB; the validated valid-map has 0 of 512,000 mismatches.

5.6 Stitching

Legacy. For every B-scan, `interp2(x,z,scan.*factor,xxAll,zzAll,'linear',0)` and the same for the weight are evaluated on the *entire* 35×6000 output plane, although one tile covers only 500 of the 6000 columns and a handful of output rows.

Port. Linear interpolation on a rectilinear grid is separable and linear, so it is written as $\text{num} += W_z (A_c \odot F) W_x^\top$ with $W_z \in \mathbb{R}^{35 \times R}$ and $W_x \in \mathbb{R}^{C \times 500}$ built by `linear_interp_matrix` (MATLAB `griddedInterpolant` semantics, 0 outside). Both are restricted to the R tile rows and C output columns that receive non-zero weight, precomputed per (x_i, z_i) tile position (96 `TileStitchers`). At run time a batch contributes via two batched matmuls for the data and two for the weights (cuBLAS on GPU, OpenBLAS on CPU), added into the column window $[c_0, c_1)$ of the accumulators. On the GPU the accumulators for a whole y -row ($500 \times 35 \times 6000 \times 2 \text{ float32} \approx 840 \text{ MB}$) stay resident in VRAM; the host only receives the finished dB row. Measured: 0.94s per 100 planes in the fused GPU benchmark (vs. 0.71s *per plane* for the legacy `interp2s`).

5.7 Thread oversubscription on the CPU

The CPU path initially measured 2.02s/plane with a stitching stage of 26s. Cause: Numba’s thread pool (16 threads) was active while every small `matmul` in the stitcher launched its own OpenBLAS thread team, so the two thread pools oversubscribed the 16 hardware threads (spin-waiting and hand-offs between pools). The stitch GEMMs are small ($R \approx \text{tens}$, $C \leq 500$), so they gain nothing from BLAS threading. Setting `OPENBLAS_NUM_THREADS=1` dropped the stitch stage from 26s to 3s and the total to 1.50s/plane. This does not affect the GPU path (cuBLAS).

5.8 Output

Legacy. Each plane is written as its own TIFF with a per-plane clim (`partialFileMode=2`), then all 4500 planes are re-read, re-quantised to the global clim and merged (`partialFileMode=3`) — two uint16 quantisations; the $\approx 100 \text{ KB}$ JSON metadata string is repeated in the `Software` tag of all 4500 pages.

Port. Rows are written into a float32 memory-mapped `.npy` (kept as `<name>_dB_float32.npy`); per-plane clim is collected on the fly; at the end one pass writes the BigTIFF (`PackBits`, identical layout) with a single quantisation to the global clim. JSON metadata is stored only on page 0 (the only page `yOCTFromTif.m` reads) and as a `.tiff.json` sidecar. `legacy_double_quantization: true` reproduces the legacy two-step rounding for bit-level comparisons. For `10um_FOV_1` the port’s TIFF is 1.906 GB versus 3.245 GB for the legacy file, mostly because the legacy file repeats the $\approx 100 \text{ KB}$ JSON on every page; writing it took 15 s.

5.9 Why not everything on the GPU?

File reading and TIFF encoding stay on the CPU: the former is limited by the USB drive and runs in parallel with GPU work; the latter is a one-off pass after the reconstruction whose cost is dominated by `PackBits` compression and disk writes. Host→device copies of int16 raw data (3.47s per 100 planes, $\approx 15\%$ of fused wall time) use pageable memory and the default stream; pinned buffers and a second CUDA stream would overlap them with compute, but the pipeline is already waiting on the disk (io_wait 12.78s of 22.65s), so this was not pursued.

6 Benchmarks

6.1 Method

`scripts/benchmark.py` reconstructs `--planes` consecutive output planes of y -row 4 (start frame 100; all 96 tiles per plane), after a 2-plane warm-up that JIT-compiles Numba/CUDA kernels and creates cuFFT plans. It reports wall time, s/plane, B-scans/s, raw MB/s, the extrapolation to 4500 planes and cumulative stage timers (`io_wait`: time the compute loop blocked on the prefetch queue; `h2d`: host→device copy; `spectral`: pre-FFT + FFT + abs; `stitch`: OPC + stitching; `finalise`). GPU timers synchronise the device, so they are exclusive. GPU benchmarks used 100 planes (9600 B-scans, 20.6 GB). A planned legacy `parfor` micro-benchmark (`validation/matlab/bench_legacy_parfor.m`) was cancelled in favour of a measured full-volume MATLAB run (§6.4); if its `LEGACY_PARFOR` log line appears, it is added to Table 6 automatically.

6.2 Throughput

Table 6: Reconstruction throughput. Full volume = 4500 planes extrapolated from the per-plane rate (except the last row). [†]Measured during development (values recorded in the project log, not in `benchmarks/results/`).

Implementation	s/plane	full volume	vs. legacy	source
Legacy MATLAB R2024a, serial (1 plane, 8 BLAS threads)	20.4	25.5 h	1.0×	docs/01 §7
Legacy MATLAB, full volume (measured)	–	OOM-killed	–	killed after 3.9 min, 8 workers
Python CPU, NumPy only (banded sinc, pocketfft)	4.14	5.2 h	4.9×	measured [†]
Python CPU, Numba fused kernel (BLAS oversubscribed)	2.02	2.5 h	10.1×	measured [†]
Python CPU, Numba fused kernel, OPENBLAS_NUM_THREADS=1	1.5	1.9 h	13.6×	measured [†]
Python CPU, full volume (measured)	1.29	1.6 h	15.8×	full_run_cpu_summary.json
GPU CuPy, unfused pre-FFT	0.393	29.5 min	51.9×	bench_gpu_b50.json
GPU CuPy + fused CUDA kernel	0.226	17.0 min	90.1×	bench_gpu_fused_b50.json
GPU fused, io_threads=8	0.215	16.1 min	94.9×	measured [†]
GPU fused, full volume (measured, incl. TIFF)	0.245	18.4 min	83.1×	run_summary.json

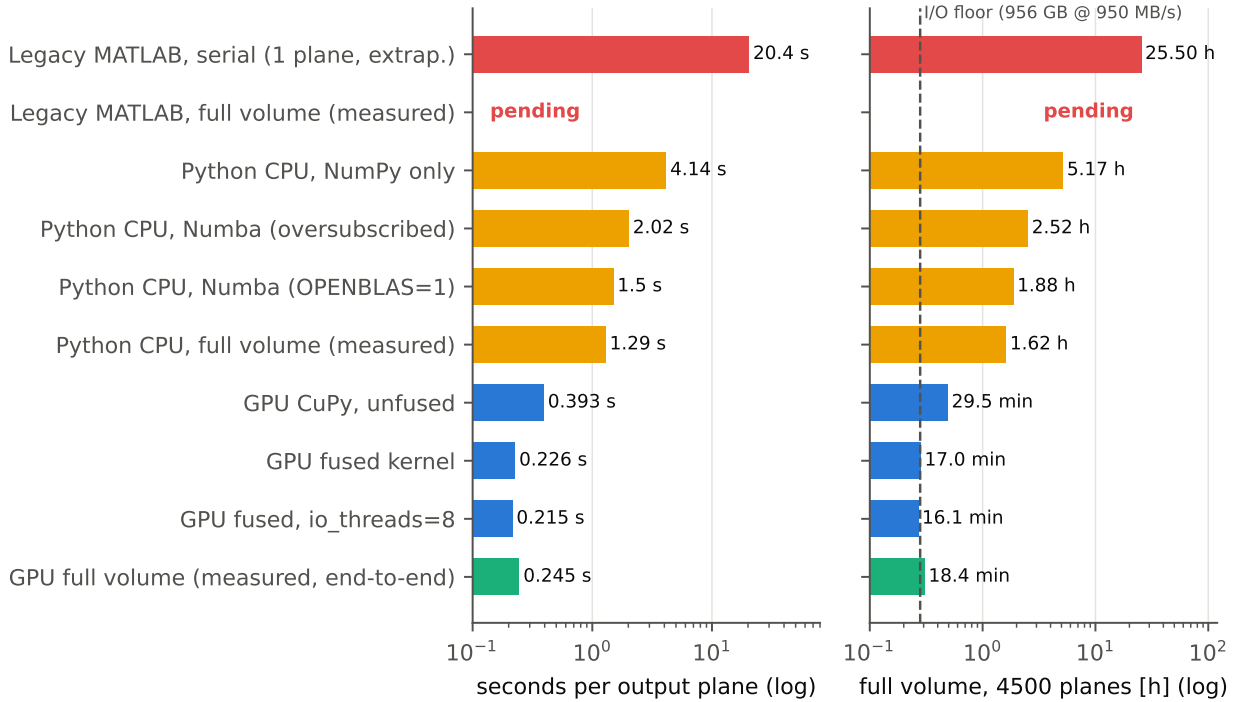


Figure 3: Seconds per output plane (left) and extrapolated full-volume time (right), log scales. Red = legacy MATLAB, orange = Python CPU, blue = Python GPU, green = actual full-volume GPU run. The dashed line is the I/O floor for reading 956 GB at 950 MB/s.

6.3 Per-stage breakdown

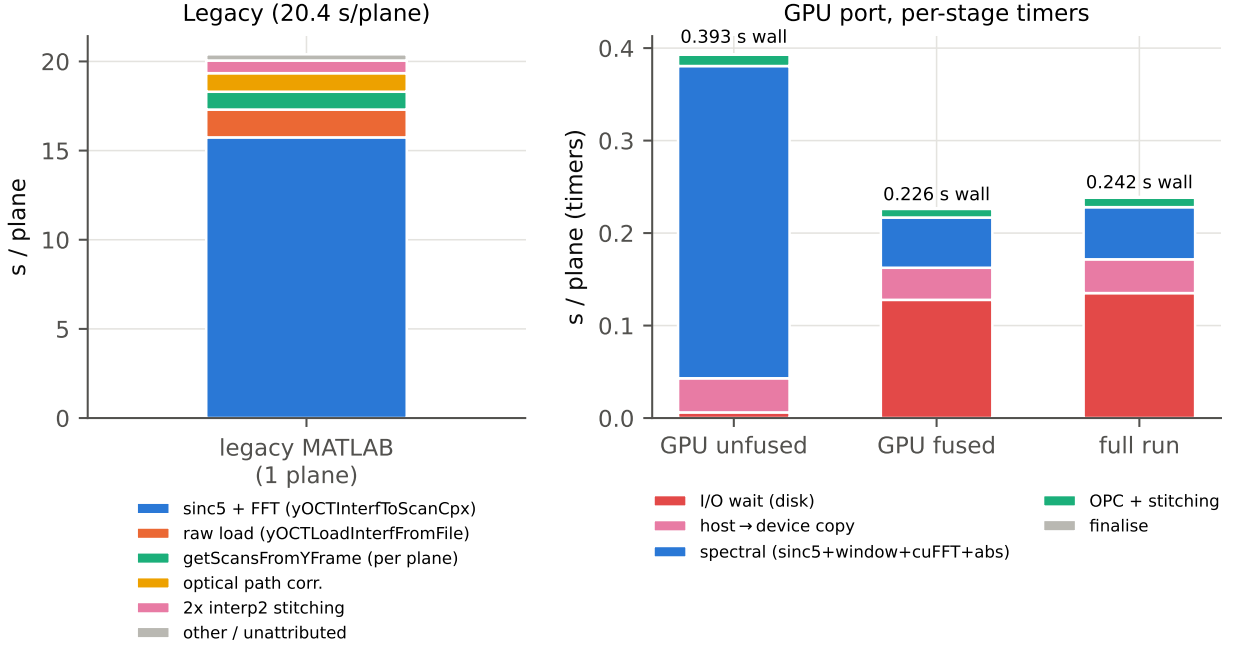


Figure 4: Left: legacy per-plane stage times (Table 4; “other” is the unattributed remainder of 20.4s). Right: GPU stage timers per plane from `bench_gpu_b50.json` (unfused) and `bench_gpu_fused_b50.json` (fused) and the full-volume run summary. Note the different y scales (factor ≈ 50).

The unfused GPU run is compute-bound (spectral stage 86 % of wall time, disk at 525 MB/s). With the fused kernel the spectral stage shrinks 6.2 $\times$ and the loop now waits on the disk for 56 % of the time while reading at 911 MB/s (424 B-scans/s). Further GPU optimisation cannot reduce wall time on this storage; increasing `io_threads` to 8 gave a last 5 % (0.215 s/plane).

6.4 Full-volume production run

The full reconstruction (`scripts/reconstruct.py configs/10um_FOV_1.yaml --device gpu`) of all 9 *y*-rows (4500 planes) took 18.2 min (1090 s, 0.242 s/plane) plus 15 s for the BigTIFF: **18.4 min end-to-end**, 83 $\times$ faster than the extrapolated legacy serial run, at an effective raw read rate of 877 MB/s. Cumulative stage timers: `io_wait` 10.1 min, `h2d` 2.7 min, `spectral` 4.2 min, `stitch` 0.8 min, `finalise` 0.0 min. The loop waited on the disk for 56 % of the time; the GPU compute stages (spectral + stitch) amount to only 0.067 s/plane (≈ 0.10 s/plane including host→device copies), i.e. well under half of the 0.242 s/plane wall time. Global clim: $[-32.87, 33.97]$ dB (legacy: $[-32.87, 33.97]$ dB).

The full-run rate (0.242 s/plane) is slightly slower than the 100-plane benchmark (0.215 s/plane): per-row times (Fig. 5) were 111–148 s with a median of 113 s; rows 4 and 5 took ≈ 148 s because a concurrent `conda install` competed for the disk and CPU. With both rows at the median time the reconstruction would have taken ≈ 17.0 min.

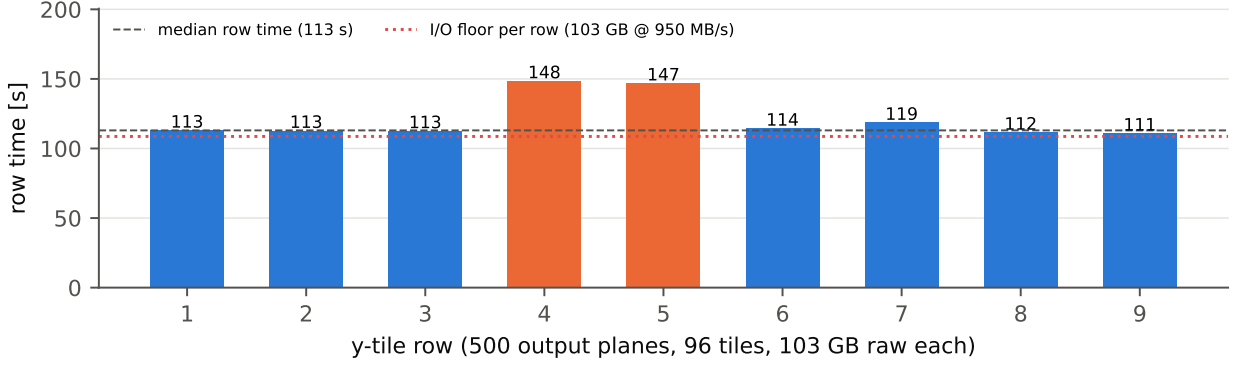


Figure 5: Wall time per y -tile row of the full GPU run (from `outputs/10um_FOV_1_full_run.log`). Orange: rows slowed by a concurrent `conda install`. Dotted: time to read one row’s 103 GB at 950 MB/s.

6.5 Full-volume runs on the CPU and in legacy MATLAB

After the GPU output had been inspected, the same full volume was reconstructed sequentially (`scripts/run_timing_baselines.sh`, nothing else running) on the CPU and with the original MATLAB code.

Python pipeline on the CPU (`--device cpu`). The CPU run took **1.6 h** end-to-end (1.29 s/plane), i.e. the GPU run is **5.3 $\times$** faster on the same code base. Its output agrees with the GPU output to within 1 LSB on all 945 M voxels (99.995 % identical codes, identical NaN masks and `clim`; `benchmarks/results/cpu_vs_gpu_validation.json`). The CPU run is *compute-bound*: 98.7 % of its time is the spectral stage (Numba `sinc5` kernel + `pocketfft`), the disk is read at only ≈ 160 MB/s on average, and all 16 hardware threads are saturated.

Table 7: Resources of the CPU full-volume run (2s samples, `full_run_cpu_resources.csv`).

	mean	p95	max
Process CPU [cores]	10.8	11.4	13.2
Hardware threads >50 % busy (of 16)	15.9	16.0	16.0
Process RSS [GB]	4.6	5.6	6.1
System RAM used [GB]	11.2	11.7	11.8
Disk read [MB/s]	160	176	838
GPU utilisation	0	0	0

Original MATLAB reconstruction. The unmodified `yOCTProcessTiledScan` was started with exactly the parameters of `configs/10um_FOV_1.yaml` (`validation/matlab/run_legacy_full.m`) on MATLAB’s default local pool (Processes profile, **8 workers**). After **3.9 min** the Linux kernel **out-of-memory killer terminated the MATLAB client** (Out of memory: Killed process ... (MATLAB) `anon-rss:1596900kB`); the 8 orphaned workers had to be terminated manually and no output was produced. Memory budget: 8 workers $\times \approx 1.1$ GB + ≈ 1.6 GB client + desktop/VS Code server exceeded the 14 GB of RAM (the machine has no swap). **The original workflow therefore cannot run at its default settings on this workstation**, whereas the port needs 3.0 GB (GPU) to 4.6 GB (CPU) resident on average. A rerun with ≤ 4 workers would fit (estimated 10–20 h, since each worker is single-threaded while the 20.4 s/plane reference used 8 BLAS threads); until then the legacy full-volume time is given by the measured single-plane extrapolation, 25.5 h serial.

6.6 Resource usage during the full run

`scripts/monitor_resources.py` sampled the reconstruction process every 2 s into `benchmarks/results/full_run_gpu_resources.csv`. 449 samples over 15.1 min (summary statistics also in `full_run_gpu_resources_summary.json`). Integrated disk reads over the monitored window: 756 GB.

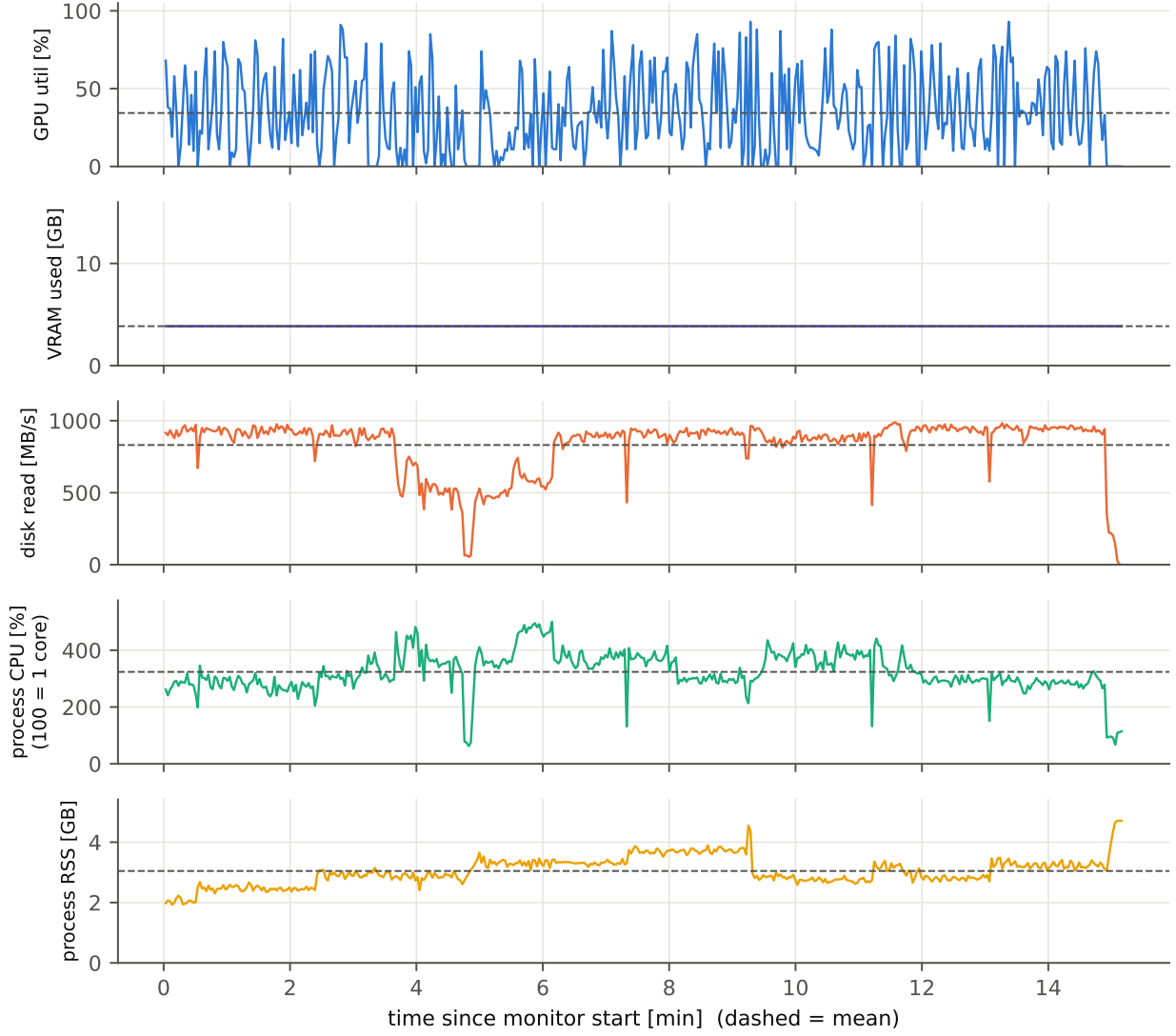


Figure 6: Resource timeline of the full-volume GPU run: GPU utilisation, VRAM, data-disk read rate, CPU use of the reconstruction process (100% = one core) and its resident memory.

Table 8: Resource statistics over the monitored window.

Quantity	Unit	Mean	Median	P5	Peak
GPU utilisation	%	34.4	31.0	0.0	93.0
VRAM used	MB	3948	3948	3948	3948
GPU power	W	36.7	37.1	32.0	41.6
GPU temperature	°C	58	58	57	59
Disk read	MB/s	831	904	467	987
Disk write	MB/s	5.4	0.0	0.0	320.4
Process CPU (100 = 1 core)	%	324	312	240	501
System CPU (all 16 threads)	%	37.0	24.0	18.2	95.2
Busiest core	%	66	59	34	100
Process threads		12	12	12	12
Process RSS	GB	3.05	3.01	2.39	4.71
System RAM used	GB	8.69	8.54	7.21	11.03

Reading of the timeline: disk throughput sits at the drive limit (p95 $\approx$ 960 MB/s) most of the time while the GPU is busy only $\approx$ 34% on average — it idles about two thirds of the time between batches, waiting for data, confirming the I/O-bound regime. VRAM stays flat at 3948 MB (CuPy memory pool: row accumulators, a few batches, precomputed operators), far below 16 GB. The process uses $\approx$ 324% CPU (3–5 cores: reader

threads plus the Python driver) and at most 4.71 GB RSS, leaving the 14 GB machine usable. The stretch of reduced read rate ($\approx 450\text{--}750$ MB/s for about two minutes) coincides with rows 4–5 and a concurrent `conda install`; the system-wide CPU peaks ($\approx 95\%$) in Table 8 were caused by that install and other concurrent processes, not by the reconstruction. The monitor was started a few minutes after the reconstruction was launched, so the window does not include the first minutes of the run.

7 Validation and image quality

7.1 Stage-by-stage against the unmodified MATLAB code

`validation/matlab/make_reference_plane.m` runs the unmodified legacy functions for output plane $y = 2250$ and dumps every intermediate (`validation/reference_data/ref_y2250.mat`); `validation/validate_stages.py` feeds the same raw B-scan through the port.

Table 9: Maximum relative error of the port vs. MATLAB per stage (single B-scan).

Stage	float64	float32
sinc5 equispaced interferogram	2.4×10^{-12}	$\approx 1 \times 10^{-6}$
complex scan (after window, dispersion, IFFT)	8.5×10^{-13}	1.5×10^{-7}
scan	1.6×10^{-13}	1.4×10^{-7}
optical-path valid map	0 / 512,000 mismatches	

7.2 Full plane (all 96 tiles, stitched)

Comparing the complete output plane $y = 2250$ in dB with the MATLAB reference (`tests/test_reconstruction.py::test_plane_matches_matlab`, identical NaN masks required):

	max $ \Delta $ [dB]	ratio to uint16 step (1.02×10^{-3} dB)
float64	1×10^{-6}	$\approx 10^{-3}$
float32 (production)	9.3×10^{-5}	≈ 0.09

The uint16 step is $(c_2 - c_1)/65534$ with the legacy `clim` $[-32.87, 33.97]$ dB. The float32 error is an order of magnitude below one output LSB, which is why float32 is the default.

7.3 Bit-exact reproduction of the legacy TIFF

The existing legacy output `10um_FOV_1/10um_H&E_1mmFOV.tiff` was produced with an unknown dispersion value (the demo script says 8.962×10^7). Reconstructing planes against it with legacy double quantisation (`legacy_double_quantization`) gives:

Table 10: Dispersion sweep: mean $|\Delta\text{dB}|$ between port and legacy TIFF.

β [nm^2/rad]	origin	mean $ \Delta $ [dB]
7.943×10^7	function default (<code>yOCTProcessTiledScan</code>)	1.62
8.6883×10^7	probe .ini (<code>DefaultDispersionQuadraticTerm</code>)	0.478
8.962×10^7	<code>Demo_ScanAndProcess3D.m</code>	0.0241
9.2×10^7	sweep point	0.455
8.949×10^7	fine sweep optimum	exact

At $\beta = 8.949 \times 10^7$ the port is bit-identical on 99.9–100 % of pixels of planes 700 and 2249, with a maximum difference of 1 LSB (rounding ties of the double quantisation). This is enforced as a regression test (`test_bit_exact_vs_legacy_tiff`: max 1 LSB, $> 99.5\%$ identical). It also establishes that the legacy TIFF used 8.949×10^7 , not the demo’s 8.962×10^7 ; the production config uses 8.949×10^7 .

7.4 Full-volume validation

The complete production output (GPU, float32, default single quantisation, $\beta = 8.949 \times 10^7$) was compared voxel-by-voxel with the legacy TIFF `10um_H&E_1mmFOV.tiff` (Table 11). All 945,000,000 voxels are within 1 LSB (max 0.00102 dB), the NaN masks (no-data regions) are identical and the global `clims` agree to

$\approx 4 \times 10^{-6}$ dB. The 90.7% identical-code rate (vs. 99.9–100% in the single-plane test) is expected: the port quantises once, directly from float dB to the global clim, whereas legacy first quantises each plane to its own clim and then re-quantises; that extra rounding step moves roughly one voxel in ten by one LSB. With `legacy_double_quantization: true` the port reproduces that step (§5.8). The port’s single quantisation is the more accurate of the two.

Table 11: Full-volume comparison of the production GPU output (`10um_FOV_1_recon.tiff`) with the legacy TIFF (`full_volume_validation.json`).

Planes compared	4500 planes of 35×6000
Voxels compared	945,000,000
NaN-mask mismatches	0
uint16 step (1 LSB)	0.00102 dB
Max $ \Delta $	0.00102 dB (1.00 LSB)
Mean $ \Delta $	0.00019 dB (0.19 LSB)
Identical codes ($ \Delta < 0.5$ LSB)	90.71 %
Within 1 LSB	100.00 %
Within 2 LSB	100.00 %
clim port	[-32.86936, 33.97096] dB
clim legacy	[-32.86936, 33.97096] dB
Worst planes (0-based)	288, 287, 281, 283, 269

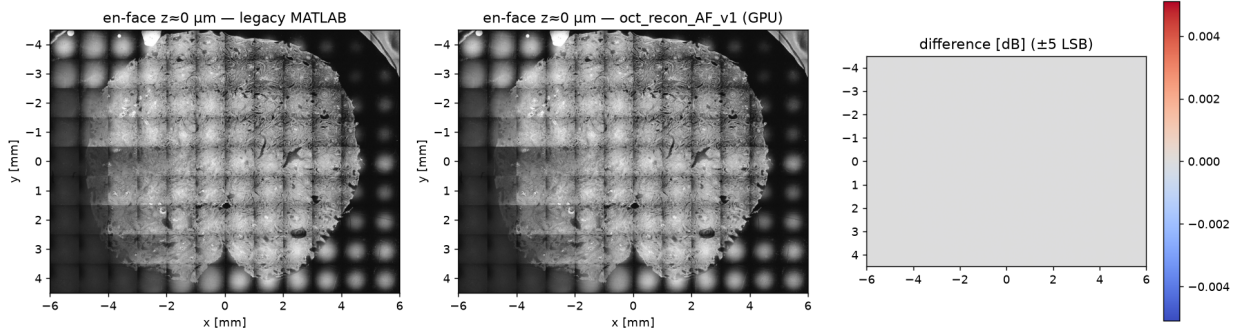


Figure 7: En-face slice at $z \approx 0 \mu\text{m}$ over the full 12×9 mm field: legacy MATLAB (left), `oct_recon_AF_v1` GPU (centre) and difference in dB (right, colour range ± 5 LSB).

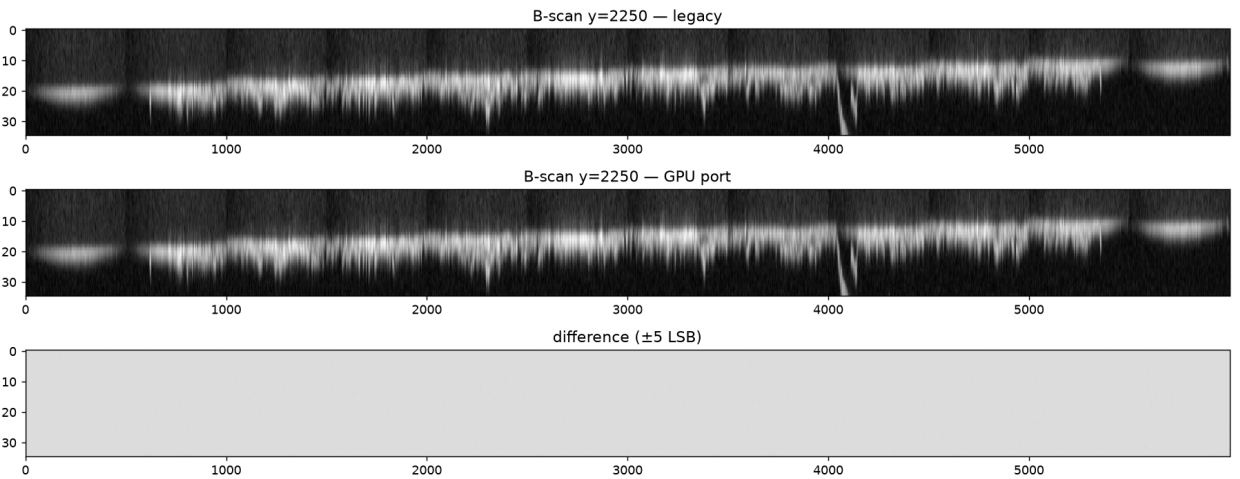


Figure 8: Output B-scan (plane $y = 2250$, 35×6000 px, axes in pixels): legacy (top), GPU port (middle), difference (± 5 LSB colour range, bottom).

8 Reconstruction parameters

The port takes exactly the inputs of `yOCTProcessTiledScan`. Items read automatically from the raw data folder: raw tiles, `Header.xml` (2048 spectral px / 525 A-lines / 25 apodization / 500 B-scans / averaging 1), chirp, `ScanInfo.json` geometry (12×9×8 tiles, 2µm, 1 mm tiles), tissue refractive index (1.33) and the optical-path polynomial. The remaining ones live in `configs/<dataset>.yaml`:

Table 12: User/calibration parameters for 10um_FOV_1 (docs/04).

Parameter (config / legacy name)	Value	Source / remark
<code>dispersion_quadratic_term / dispersionQuadraticTerm</code>	8.949×10^7 nm ² /rad	reproduces the legacy TIFF bit-exactly (§7); not stored with the raw data. Critical: a 0.1 % change is visible at LSB level, 3 % visibly blurs.
<code>focus_positions / focusPositionInImageZpix</code>	433 (all 8 depths)	<code>zChosenFocusPositions.mat</code> from the interactive <code>yOCTMeasureFocusDrift</code> (one click at $z = 0$; no drift).
<code>focus_sigma / focusSigma</code>	10 px	per objective; weight $e^{-(z-f)^2/(2\sigma)^2} \Rightarrow$ effective std $\sqrt{2}\sigma \approx 14 \text{ px} \approx 20 \mu\text{m}$.
<code>interp_method</code>	<code>sinc5</code>	only method ported (pchip not implemented).
<code>crop_z_range_mm</code>	<code>[-0.03, 0.04]</code>	= <code>[min, max]</code> of <code>zDepths</code> .
<code>output_pixel_size_um</code>	2	must equal <code>pixelSize_um</code> (legacy restriction).
<code>n_medium</code>	<code>null</code> → 1.33	<code>ScanInfo.tissueRefractiveIndex</code> .
<code>apply_path_length_correction</code>	<code>true</code>	
<code>λ range (code constant)</code>	796–1010 nm	hard-coded for gan632 in <code>geometry.py:LAMBDA_RANGE_NM</code> , marked “TODO measured” in legacy.

8.1 What can be estimated automatically

- **Dispersion β .** `Applications/yOCTScanGlassSlideToFindFocusAndDispersionQuadraticTerm.m` runs `fminsearch` over β maximising the peak of the x -averaged log A-scan. Porting this as a bounded 1-D sharpness search on a few B-scans (one FFT per candidate) takes seconds on the GPU; it should recover $\approx 8.949 \times 10^7$. Best candidate for automation.
- **Focus pixel.** `Processing/yOCTFindFocusTilledScan.m` finds the brightest layer near $z = 0$ and refines it on a gel-only depth (`findpeaks`); runs offline on existing scans with `manualRefinement=false`. Easy port; validate against 433.
- **Focus drift / tissue RI.** `yOCTMeasureFocusDrift_fitDrift` (Theil–Sen fit) is the only in-code RI estimator; meaningful only for deep scans (not this 40µm scan).
- **focusSigma:** no estimator (per-objective constant). **cropZRange:** derivable from `zDepths`.

8.2 z-scale caveat

The depth pixel (1.4339 µm in tissue) follows from the hard-coded $\lambda \in [796, 1010]$ nm, copied from a Ganymede calibration. `Header.xml` of this dataset reports a central wavelength of 880 nm and bandwidth of 239 nm (Thorlabs’ own `RangeZ` of 1.6558 mm is consistent with that), implying a $\approx 15\%$ different z pixel size. The port reproduces legacy, so both are affected equally; correcting it would also shift the optimal β and requires a calibration (glass-slide stage sweep or a target of known thickness).

9 Code layout, usage, limitations

9.1 Folder structure

```

Q3_OCT_reconstruction/
|-- README.md, environment.yaml          conda env "oct_reconstruction"
|-- docs/                                01 legacy pipeline, 02 raw format, 04 parameters
|-- octrecon/                             deployable Python package
|   |-- backend.py                       NumPy/CuPy selection (xp)
|   |-- pipeline.py                      ReconConfig + Reconstructor (driver, prefetch, timers)
|   |-- io/ scaninfo.py thorlabs.py tiff_writer.py
|   |-- core/ geometry.py spectral.py stitching.py cpu_kernels.py gpu_kernels.py
|-- configs/<dataset>.yaml                one reconstruction config per dataset
|-- scripts/                             reconstruct.py, benchmark.py, monitor_resources.py

```

```

|-- tests/                                pytest regression tests (vs MATLAB + legacy TIFF)
|-- validation/ matlab/ reference_data/ validate_stages.py
|-- benchmarks/results/                  benchmark JSON, logs, resource CSV
|-- outputs/<dataset>/                    <name>.tiff, .tiff.json, _dB_float32.npy, _run_summary.json
|-- report/oct_recon_AF_v1/              this report
|-- <dataset>/OCTVolume/                 raw data (read-only)
`-- Reconstruction_code_legacy/          original MATLAB (read-only reference)

```

9.2 How to run

```

source ~/miniforge3/bin/activate oct_reconstruction
python scripts/reconstruct.py configs/10um_FOV_1.yaml          # full volume (GPU if available)
python scripts/reconstruct.py configs/10um_FOV_1.yaml --rows 4 # one y-tile row (500 planes)
python scripts/benchmark.py --device gpu --planes 100         # throughput test
OPENBLAS_NUM_THREADS=1 python scripts/benchmark.py --device cpu --planes 25
pytest -q tests                                                # CPU regression tests
OCT_DEVICE=gpu pytest -q tests                                 # same on GPU

```

New dataset: copy `configs/10um_FOV_1.yaml`, change `volume_folder`, `output_dir`, `output_name`, `dispersion`, `focus` and `crop` parameters (§8). Outputs: `<name>.tiff` (legacy-compatible BigTIFF), `<name>.tiff.json`, `<name>_dB_float32.npy` and `<name>_run_summary.json` (config, timers, clim).

9.3 Limitations

- **I/O bound.** With the fused kernel the GPU idles most of the time; wall time is set by reading 956 GB over USB (≈ 16 – 17 min floor).
- Only `sinc5/sincN` k-linearisation is ported (no `pchip`); no A-scan/B-scan averaging or spectral binning (raises `NotImplementedError`); a missing/truncated raw file raises instead of producing NaNs as in legacy.
- Dispersion and focus still come from manual/interactive steps (config + `.mat`).
- The absolute z scale inherits the uncalibrated `gan632` λ range ($\approx 15\%$ uncertainty).
- Host $\rightarrow$ device copies are synchronous from pageable memory (not a bottleneck today).

9.4 Next steps

1. **Faster storage.** Copy raw data to internal NVMe (3–7 GB/s) or read from it directly; the GPU has ≈ 2 – $3\times$ headroom (spectral + stitch ≈ 0.07 s/plane of compute in the fused benchmark). Alternatively store raw data compressed (12-bit values in `int16` leave 4 unused bits; lossless packing/compression reduces bytes read) and decompress on the GPU.
2. **Automatic dispersion and focus** (ports of the glass-slide β search and `yOCTFindFocusTilledScan`) so a reconstruction needs only the volume folder + a per-probe config.
3. **z-scale calibration** of the `gan632` λ range (glass-slide stage sweep).
4. Pinned host buffers + a copy stream to overlap H2D with compute once storage is faster.

A Legacy per-script reference

Paths relative to `Reconstruction_code_legacy/myOCT/` (from docs/01 §3).

File	Role	Key details
<code>Processing/yOCTProcessTiledScan.m</code>	Driver / stitcher	Defaults: dispersion 79430000, focusSigma 20, output pixel 1 μm , path-length correction on. Unmatched options forwarded to loader/FFT. <code>parfor</code> over output y planes.
<code>Processing/yOCTProcessTiledScan_crateDimStructure.m</code>	Geometry	Tile x/y = offset + <code>range.linspace(-.5, .5, $n + 1$)</code> (last dropped); galvo shift only if correction field absent; tile z zeroed at focus; output grid spans all tile centres.
<code>Processing/yOCTProcessTiledScan_getScansFromYFrame.m</code>	y plane $\rightarrow$ tiles	Tiles with matching <code>gridYcc</code> in <code>octFolders</code> order; local frame = $yI - 500(\text{row} - 1)$.
<code>Processing/yOCTProcessTiledScan_factorZ.m</code>	Focus weight	$e^{-(z-f)^2/(2\sigma)^2} + e^{-4.5}$ (note $(2\sigma)^2$).
<code>LoadSave/yOCTLoadInterfFromFile.m</code>	Loader	Thorlabs path; frame subset; apodization subtraction (mean of 25 apod. A-lines).

File	Role	Key details
LoadSave/yOCTLoadInterfFromFile_ThorlabsHeader.m	Header.xml	Sizes, interfSize 525, apodSize 25, averaging; <code>xml2struct</code> via <code>imageDatastore</code> .
LoadSave/yOCTLoadInterfFromFile_ThorlabsHeaderLambda.m	λ axis	$\lambda = 1/(\text{chirp}(1/\lambda_{\max} - 1/\lambda_{\min})/(N - 1) + 1/\lambda_{\min})$; gan632 uses 796–1010 nm (TODO in code).
LoadSave/yOCTLoadInterfFromFile_ThorlabsLoadChirp.m	Chirp	<code>Chirp.data</code> : text parse, falls back to float32 binary.
LoadSave/yOCTLoadInterfFromFile_ThorlabsData.m	Raw read	int16 'short', reshape [2048 525]; new <code>imageDatastore</code> per file.
LoadSave/yOCTLoadInterfFromFile_ReadFile.m	Safe read	Missing/corrupt file $\Rightarrow$ NaN + warning.
Processing/yOCTEquispaceInterf.m	k-linearisation	$k_{\text{lin}} = \text{linspace}(\max k, \min k, N)$; sincN: MATLAB loop over 2048 outputs, taps $ nq - n < N$, mean removed/re-added; λ of new samples via <code>pchip</code> .
Processing/yOCTInterfToScanCpx.m	Spectrum $\rightarrow$ depth	Hann/RMS, dispersion $-\beta(k - \bar{k})^2$, <code>ifft</code> , keep $N/2$; options via <code>eval</code> .
Processing/yOCTInterfToScanCpx_getZ.m	Depth axis	$dz = \frac{1}{2} \lambda_0^2 / \Delta \lambda / n = 1.4339 \mu\text{m}$ ($n=1.33$).
Processing/yOCTOpticalPathCorrection.m	Field curvature	$P = p_1x + p_2y + p_3x^2 + p_4y^2 + p_5xy$ (μm); <code>interp2</code> 'nearest' at $z + P$; out of range $\Rightarrow$ invalid.
LoadSave/yOCT2Tif.m, yOCT2Tif_ConvertBitsData.m	Output	uint16 = $\text{round}((\text{dB} - c_1)/(c_2 - c_1) \cdot 65534) + 1$, NaN $\rightarrow$ 0; BigTIFF, PackBits, JSON in tag 305; mode 2 per-plane <code>clim</code> , mode 3 re-quantises.
LoadSave/yOCTFromTif.m	Reader	JSON from first page's <code>Software</code> tag.
yOCTMeasureFocusDrift.m + _fitDrift.m	Focus (GUI)	Clicks + Theil–Sen drift fit $\rightarrow$ per-depth focus, RI; here constant 433.
Processing/yOCTFindFocusTiltedScan.m	Focus (auto)	Automatic alternative (not used for this dataset).

Known legacy quirks kept for fidelity: $(2\sigma)^2$ in `factorZ`; options parsed with `eval`; header/JSON/chirp re-read per plane; double uint16 quantisation (optional in the port); `clim` from min/max (a $-\infty$ would break scaling, not observed); placeholder gan632 λ range.

B Raw data format

B.1 Folder layout

```

10um_FOV_1/
|-- OCTVolume/                raw tiled scan (956 GB)
|   |-- ScanInfo.json          scan description (written once, after all tiles)
|   |-- zChosenFocusPositions.mat focus pixel per depth (+ .png diagnostic)
|   |-- Data01/ ... Data864/    one folder per tile (x, y, z-depth)
|       |-- Header.xml          Thorlabs OCITY header (~130 KB)
|       |-- data/ Chirp.data OffsetErrors.data Spectral0.data ... Spectral499.data
|-- 10um_H&E_1mmFOV.tiff        legacy reconstruction (uint16, 3.2 GB)
`-- Reslice of 10um_H&E_1mmFOV.tif

```

Tile index: folder (1-based) $= z_i + 8x_i + 96y_i + 1$ (0-based indices); grid order z fastest, then x , then y .

B.2 Key ScanInfo.json fields

Field	Value	Meaning
<code>octSystem</code>	gan632	selects λ range / loader
<code>pixelSize_um</code>	2	lateral pixel
<code>tissueRefractiveIndex</code>	1.33	scales depth axis
<code>xRange_mm, yRange_mm</code>	$[-6, 6], [-4.5, 4.5]$	12×9 mm area
<code>tileRangeX/Y_mm, n[XY]PixelsInEachTile</code>	1 mm, 500×500	tile size
<code>xCenters_mm, yCenters_mm</code>	$-5.5 \dots 5.5$ (12), $-4 \dots 4$ (9)	tile centres
<code>zDepths</code>	$-0.03 \dots 0.04$ step 0.01 (8)	stage z offsets; 0 = focus at interface
<code>gridXcc/Ycc/Zcc, octFolders</code>	864 entries	tile centre / folder per grid entry
<code>galvoPhaseDelayXoffsetCorrection_mm</code>	0.01915	galvo lag compensated at acquisition
<code>octProbe</code>	20x OCTG WINTER ini	optical-path polynomial, default dispersion 8.6883e7

B.3 Binary files

File	dtype / shape	Notes
Spectral{i}.data	int16 LE, 525×2048 (A-line major), 2 150 400 B	B-scan i = tile-local y frame; A-lines 0–24 apodization; 12-bit values
Chirp.data	float32[2048]	pixel $\rightarrow k$ mapping
OffsetErrors.data	float32[2048]	not used

Header.xml: 2048 spectral px, 525 A-lines (apodization 0–25, scan 25–525), 500 B-scans, 2 bytes/px; image $1024 \times 500 \times 500$ px, $1.656 \times 1 \times 1$ mm; averaging 1; 5.56 s per tile; GAN632, 12-bit camera, central λ 880 nm, bandwidth 239 nm, 100 kHz line rate.

B.4 Output format (legacy and port identical)

BigTIFF, 4500 pages (y from -4.5 to 4.498 mm), each 35 (z) $\times$ 6000 (x) uint16, PackBits. $\text{dB} = (\text{bits} - 1)(c_2 - c_1)/65534 + c_1$, bits = 0 $\Rightarrow$ NaN. Tag 305 holds JSON {"metadata", "clim", "version": 3}; the port writes it on page 0 plus a .tiff.json sidecar, and keeps the float32 dB volume as <name>_dB_float32.npy (y, z, x).